



Serverless Elasticsearch: the Architecture Transformation from Stateful to Stateless



Inès Potier

Senior Software Engineer at Elastic

Elasticsearch

What is Elasticsearch?

- Distributed, H/A, search & analytics engine
 - Ranked results (e.g., BM25), fuzzy matching
 - Spelling, Synonyms, Phrases, Stemming
 - Timeseries, geospatial
 - Vector search, (G)AI/ML, RAG search apps
- REST API
- Built on top of the Apache [Lucene](#) library

github.com/elastic/elasticsearch

 **elasticsearch** Public

Free and Open Source, Distributed, RESTful Search Engine

 Java  75.4k  25.6k

Deployment options:



Public CSP



Google Cloud



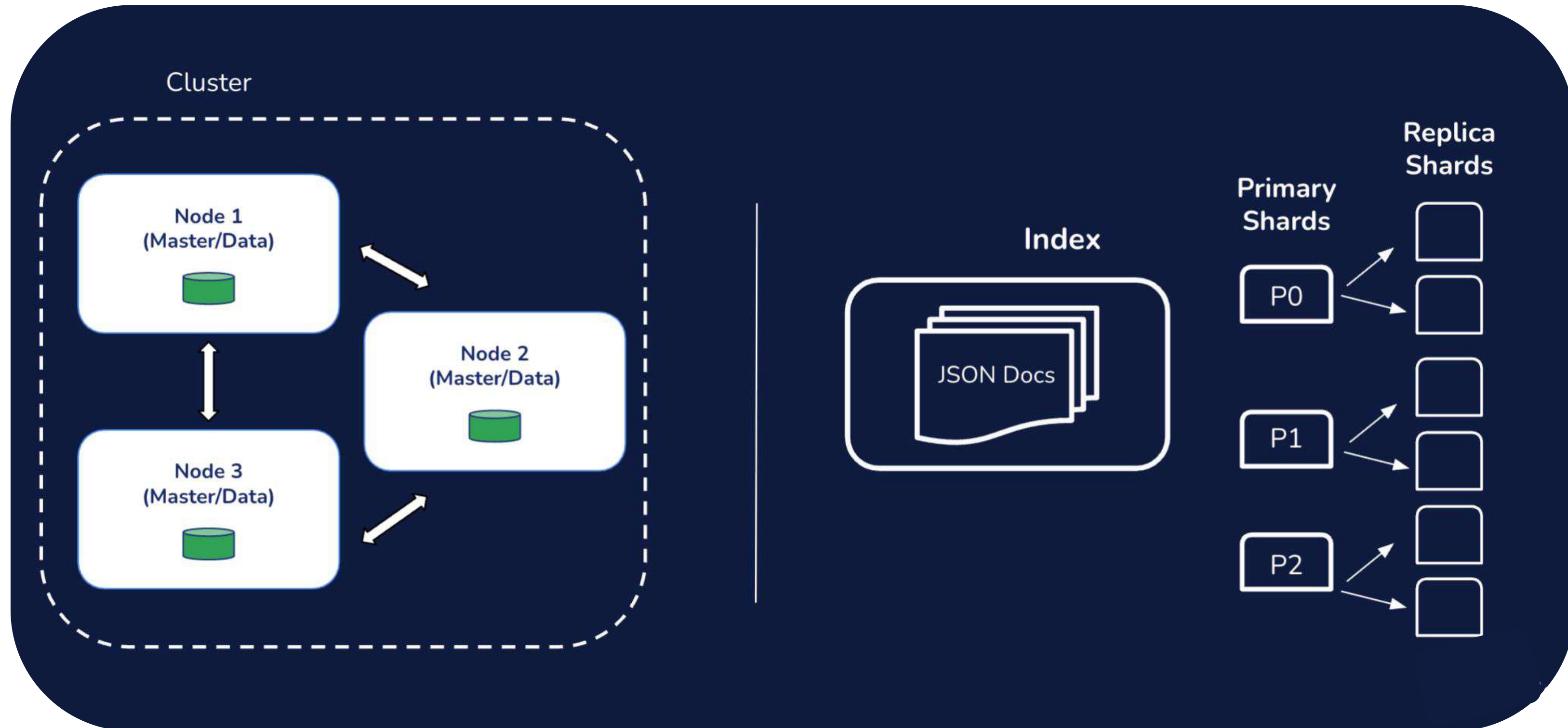
Hybrid



On-Premises

Elasticsearch

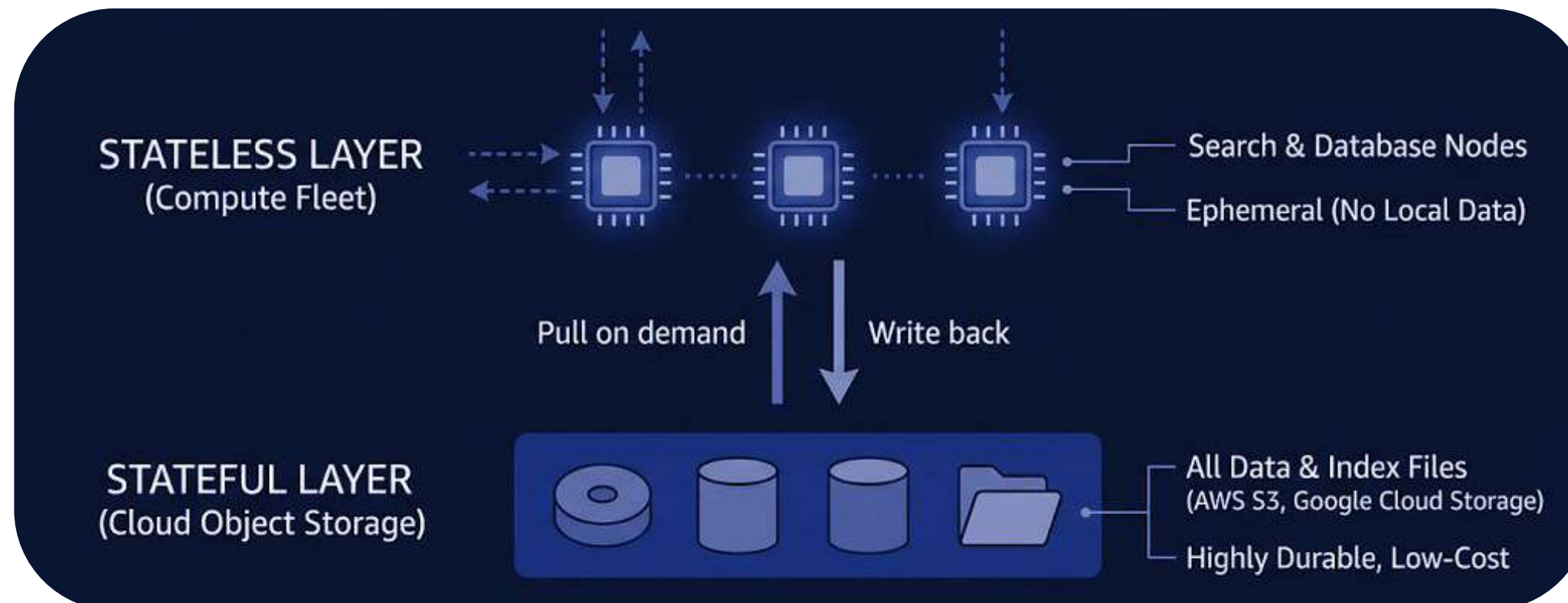
Basic Architectural Principles



What are stateless architectures?

Compute tier does not hold onto persistent data locally.

- **Stateful Layer (Cloud Object Storage):** High-durability, low-cost object stores. All data and index files live here.
- **Stateless Layer (Compute Fleet):** Search and database nodes run as pure, ephemeral compute. They pull what they need into local working memory/cache, do the work, and write results back.



Why stateless?

Stateless architectures are mainstream

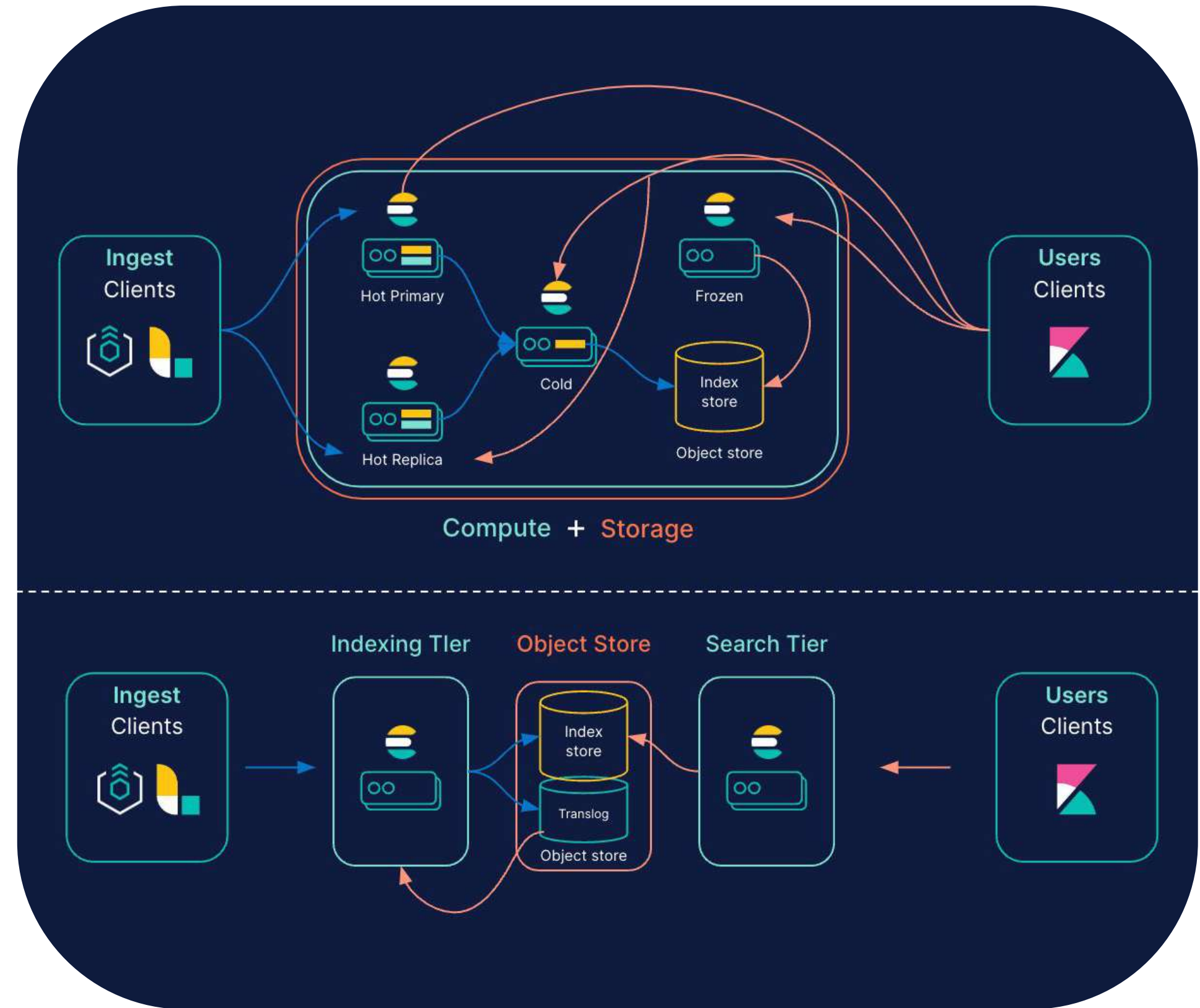
- **ClickHouse:** Storage–compute separation ([~2021](#))
- **Grafana Loki:** stateless-by-design

Key Architectural Advantages

Decoupled Compute & Storage: Scale ingest and query resources independently based on workload demand.

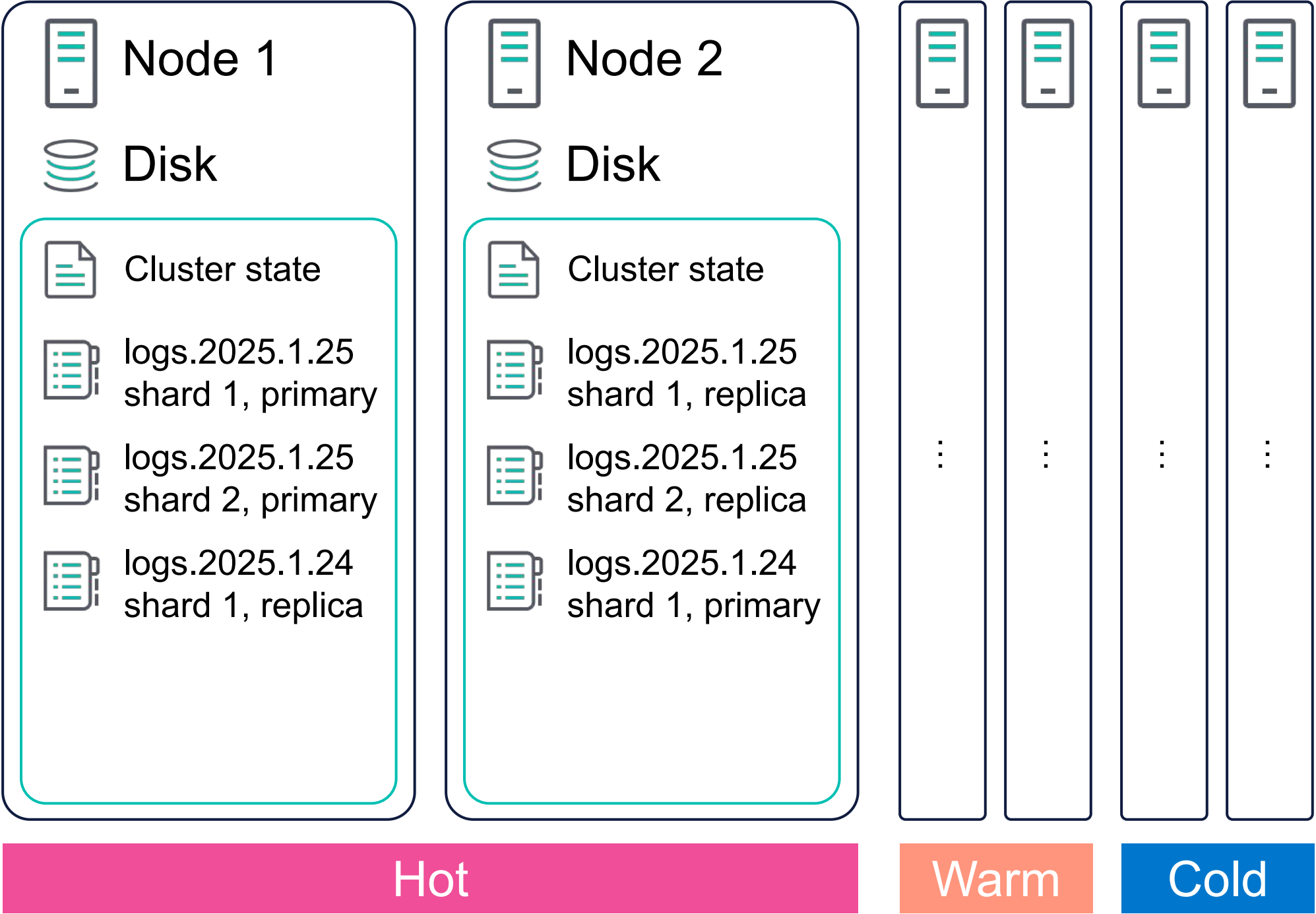
Object Store as Ground Truth: Use durable, low-cost cloud storage (S3/GCS) instead of expensive attached disks.

No-Rebalance Scaling: Spin up or down compute nodes without waiting for gigabytes of shard data to transfer across the cluster.



Elasticsearch

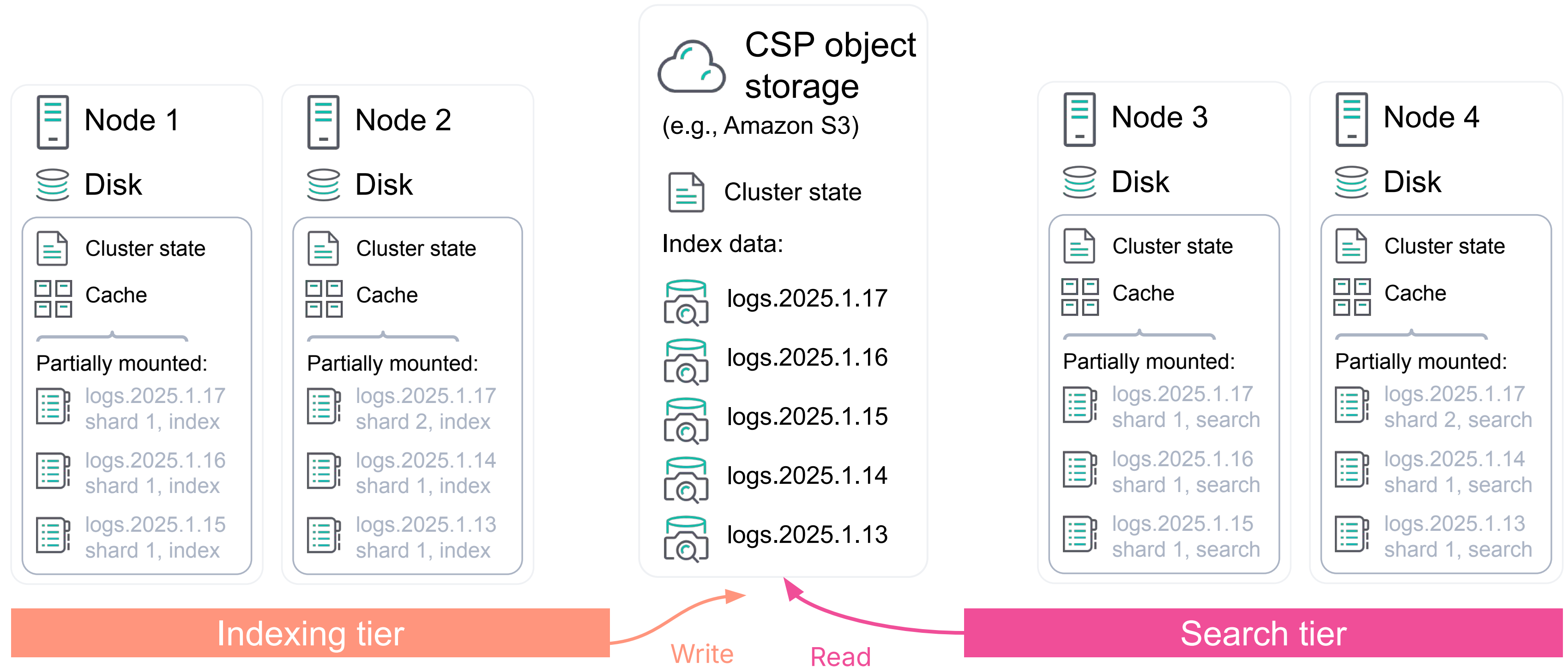
Stateful Architecture



(more performant, expensive) **Data tiers** (less performant, cheaper)

Elasticsearch

Stateless Architecture



From stateful fleets to stateless workloads

Traditional self-managed ES is optimized for disks



Stateful

- Manual data tiers & policies.
Hot → warm → cold → frozen is powerful but complex.
- CPU/RAM coupled with storage
- Ingestion & search share CPU
- Durability = replicas + disk-heavy relocate / recover
- Scaling relocates data physically

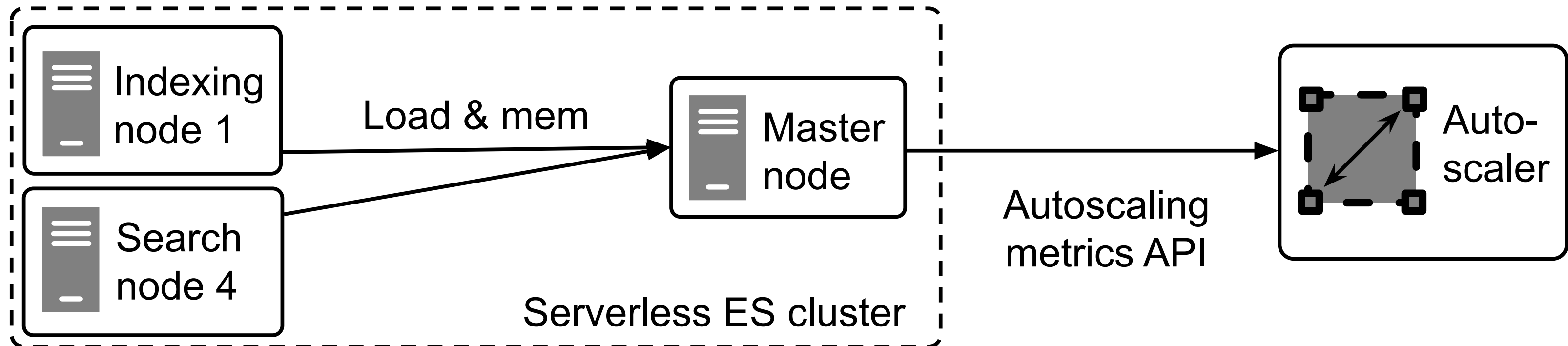


Stateless

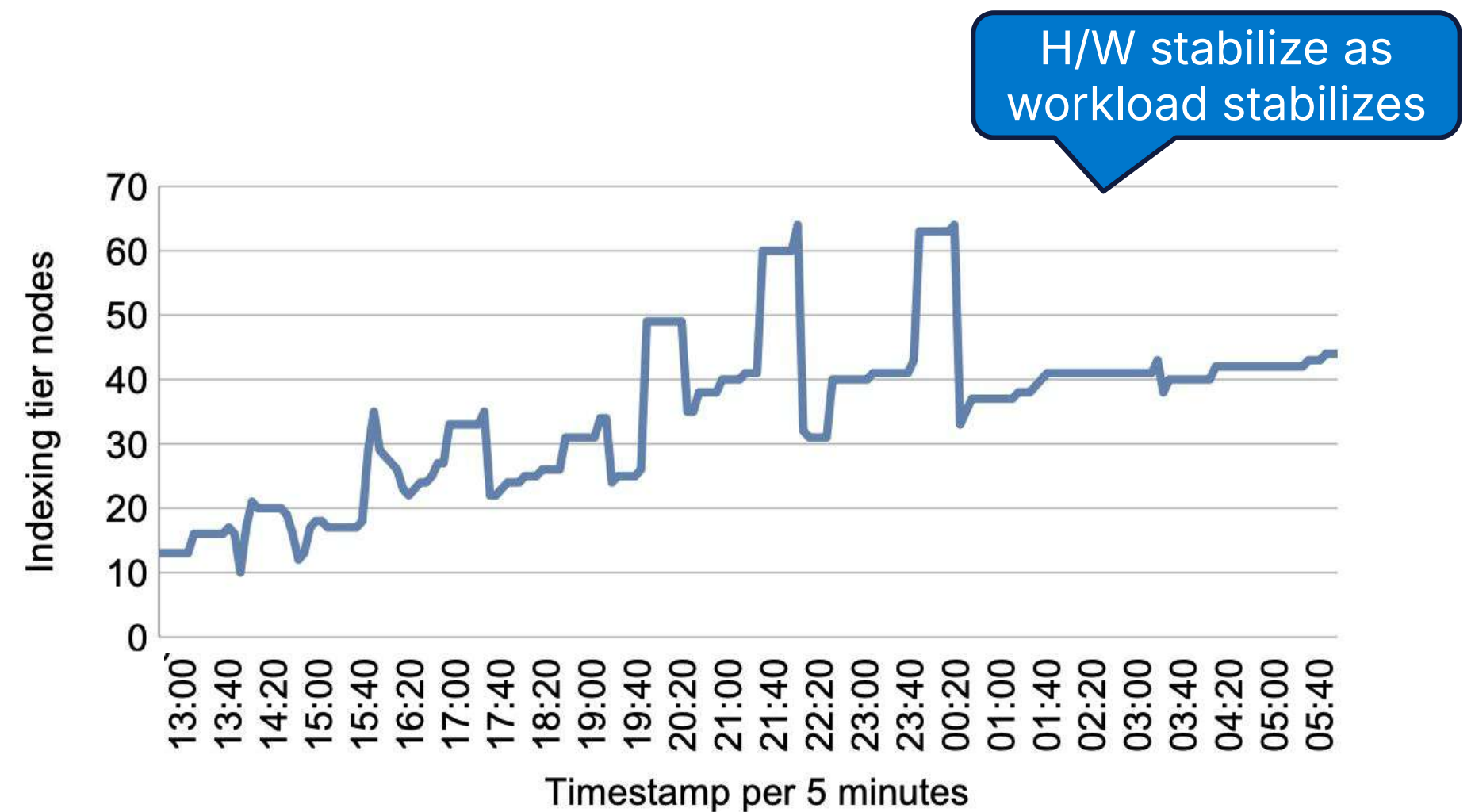
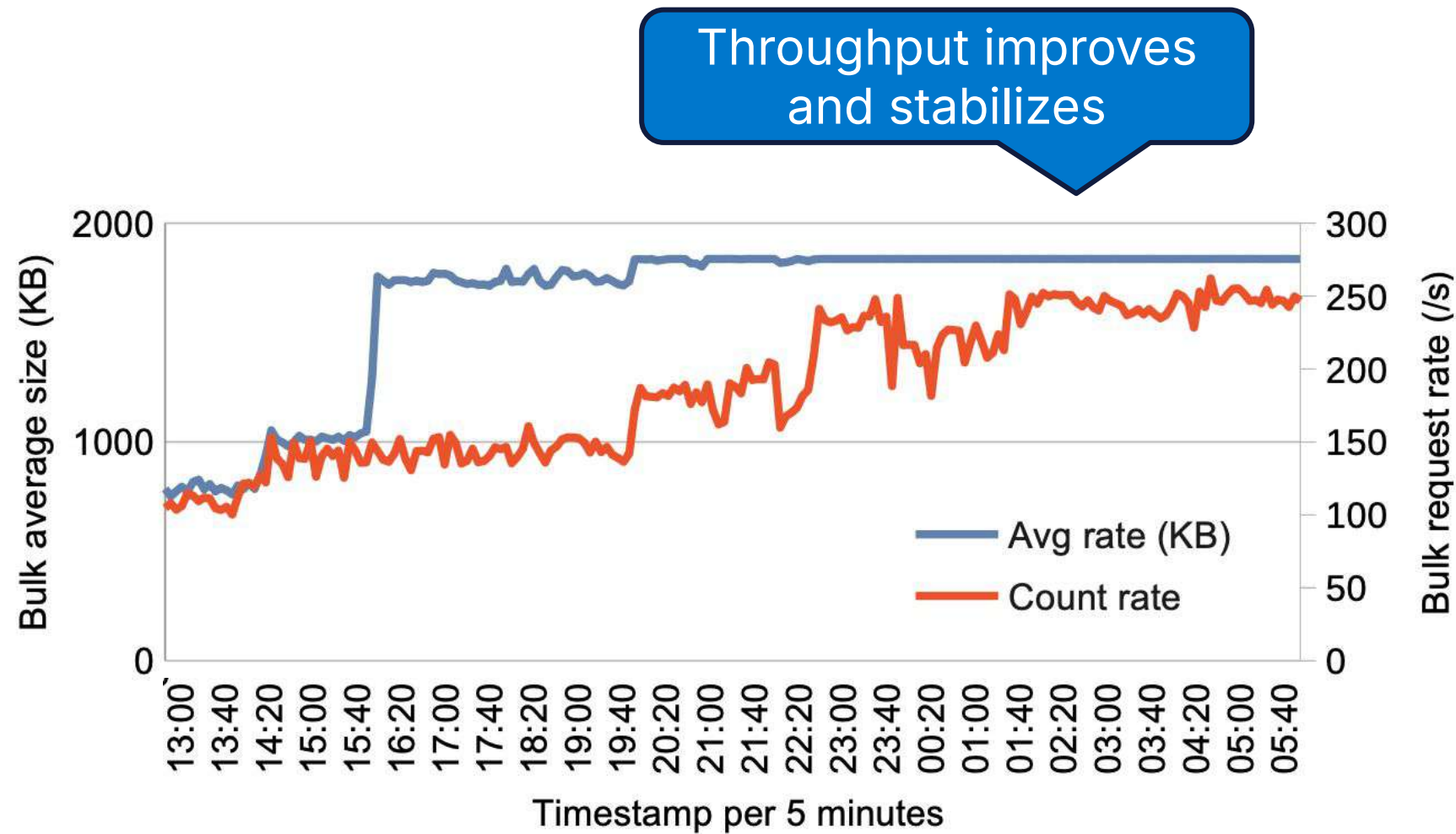
- Data in object store
- Exploits CSP resilience/scalability
- 2 independently scalable data tiers:
indexing, search
- Decouples compute from storage
- Thin shards (disk is primarily used for partially caching commit files)
- Scaling relocates thin shards fast
- Keeps same APIs mostly

New: Easier Autoscaling

- Autoscaler adjusts cluster up/down to match search and ingestion load
- Nodes report:
 - Estimated minimum heap memory
 - CPU load & queuing, reflects I/O



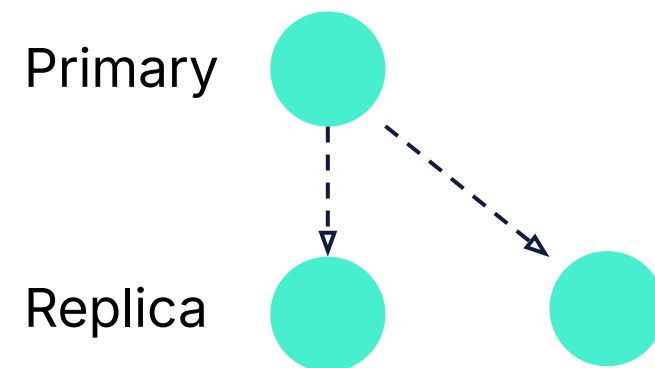
Real-world example of ingest autoscaling



From old operation-based to new segment-based replication

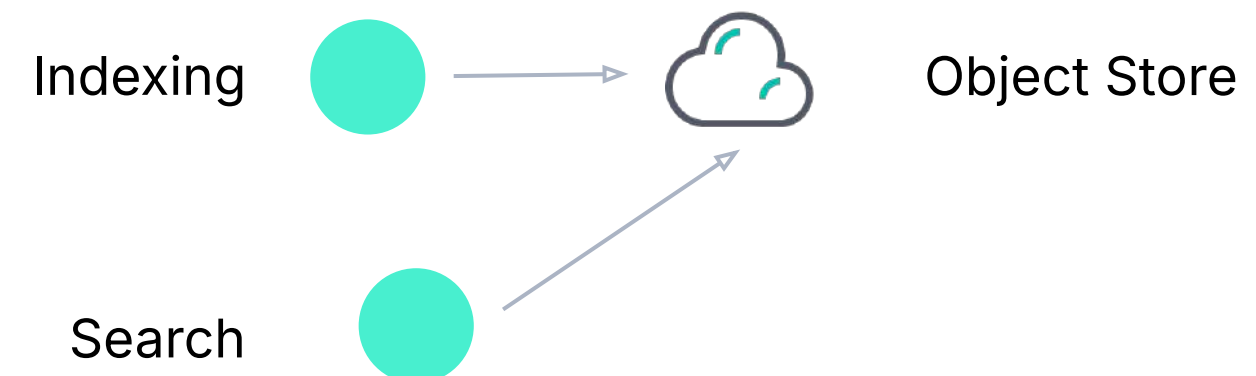
➔ Stateful

- Primary shard applies operation
- Replica applies operation (2× effort)
- A refresh exposes segments ("batches") of operations to searches



➔ Stateless

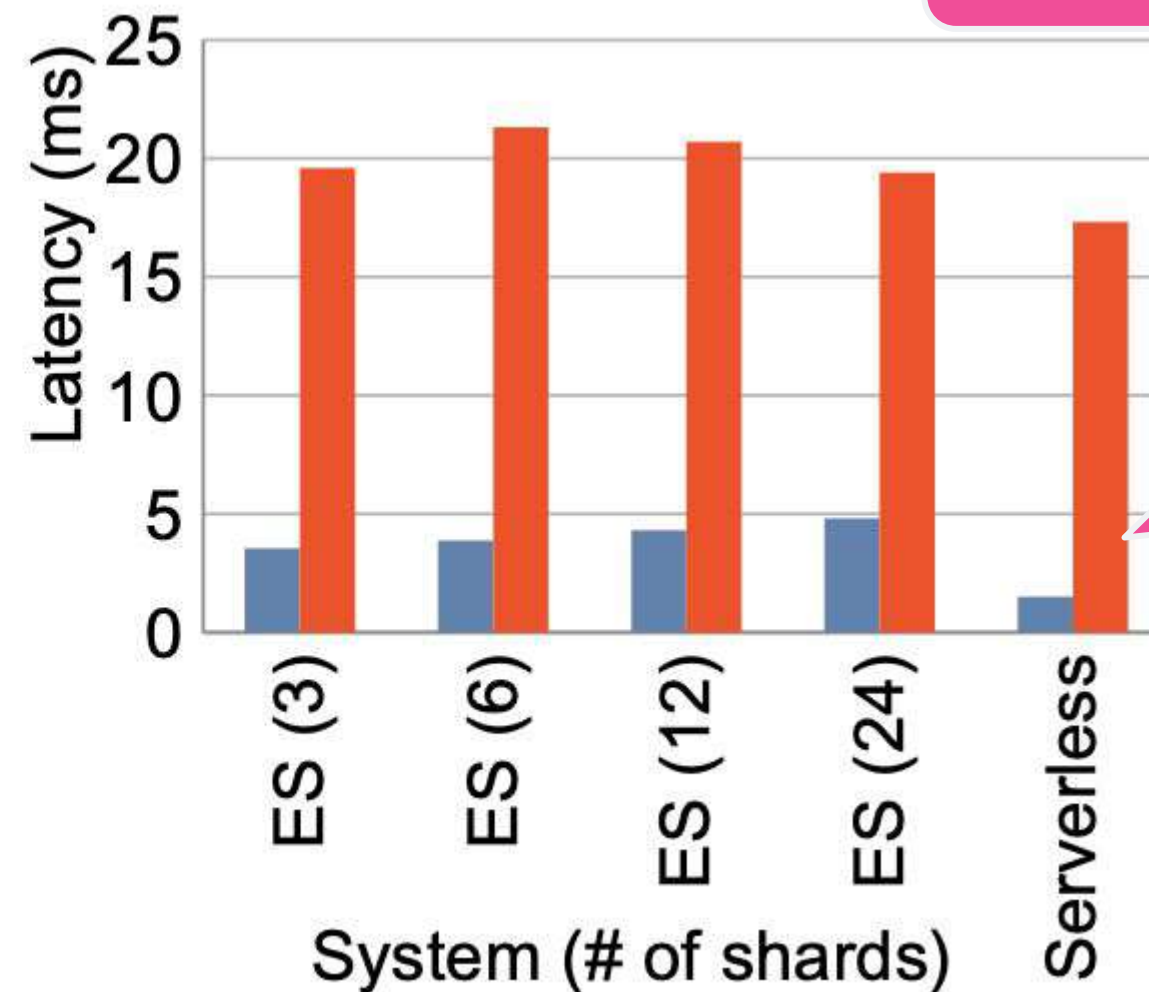
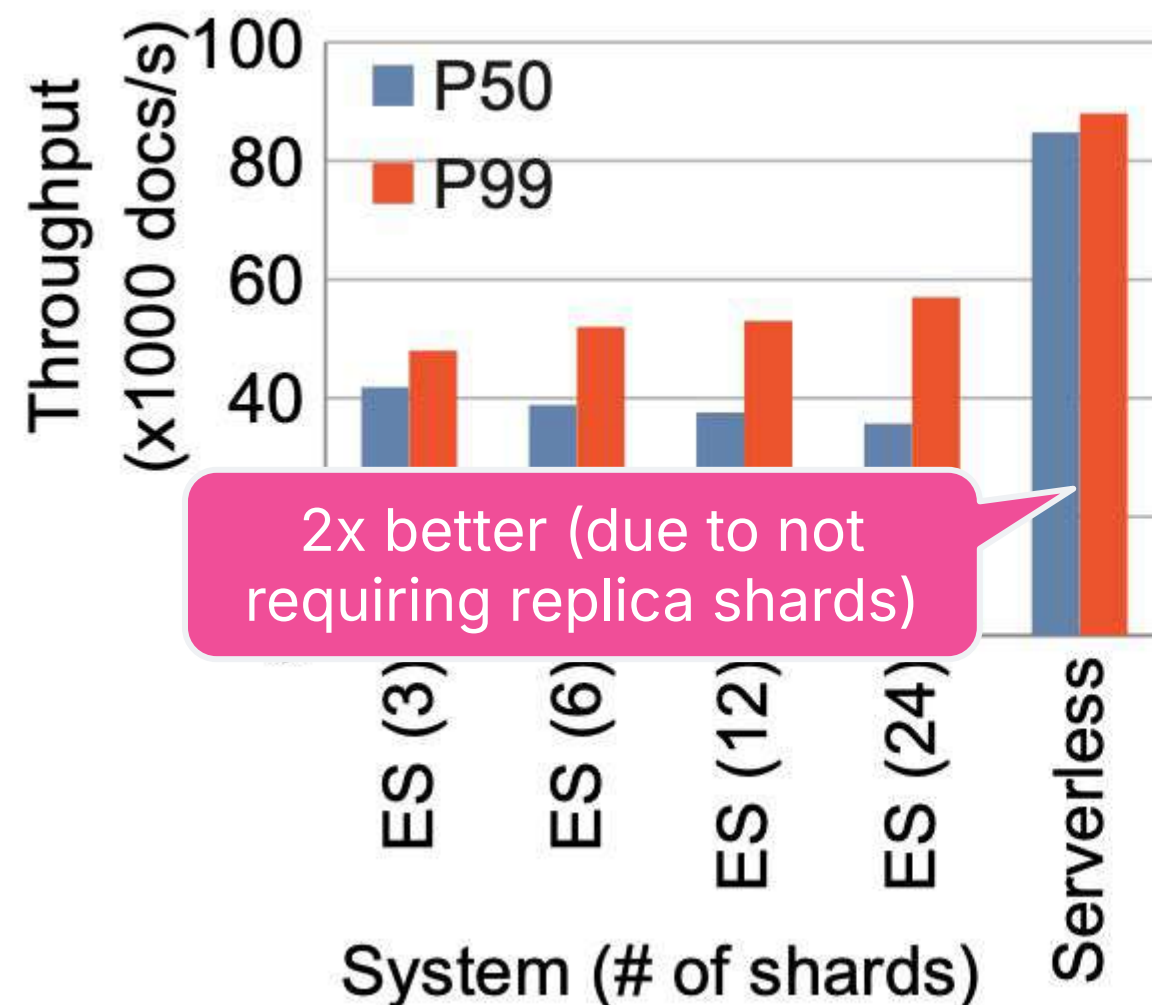
- Indexing shard uploads committed segments to the object store
- Up to 2× performance due to no replicas
- Searches get segments from object store



New: Better performance

Stateless achieves up to 2x indexing throughput as shown in our Serverless paper

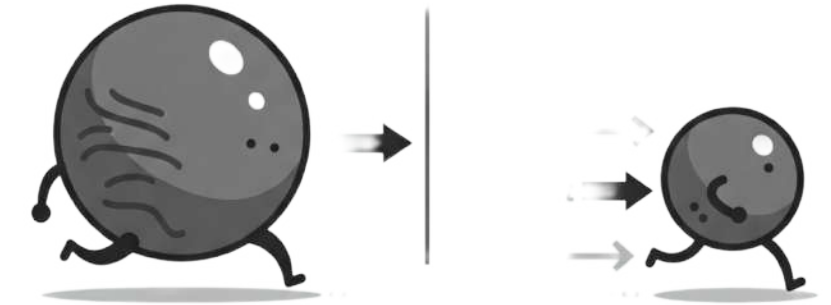
- Data ingested: 1560GB of github-archive, 96 clients, 2000 docs/request
- ES: 3×60GB RAM, 32 cores. Shards: 3, 6, 12, and 24 primaries. 1 replica.
- Serverless ES: 3×64GB RAM, 32 cores. 1 data stream (auto-sharding).



In self-managed stateless, customers will define #shards

Latency is comparable/better

New: Thin shards



- Loaded with metadata until data is indexed/searched
- Indexing generates files, uploads them and deletes them when unused
- Searching loads from object store only the required data into the cache
- Not dependent on disk, not restricted by disk capacity → stateless
 - Durability is guaranteed by the object store
 - Disk is just a cache
- Shard relocation (and recovery) is very fast
 - Indexing shard has a hand-off phase to transition workload
 - Removing/adding nodes is quick → autoscaling

Conclusion

- **Serverless ES** is stateless, decoupling compute from storage
 - Data offloaded to a CSP resilient **object store**
- Just two **tiers: indexing and search** → independent workloads
- Same API and read-after-write semantics as ES
- Hands-free workload-based **autoscaling**
- Experimental evaluation: Up to 2x better indexing throughput

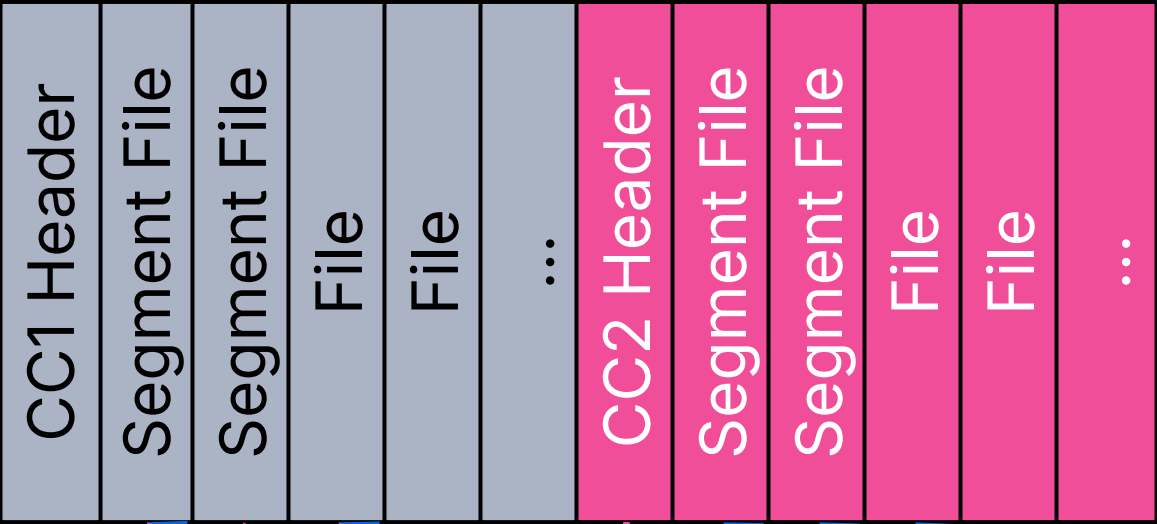
Thank You!



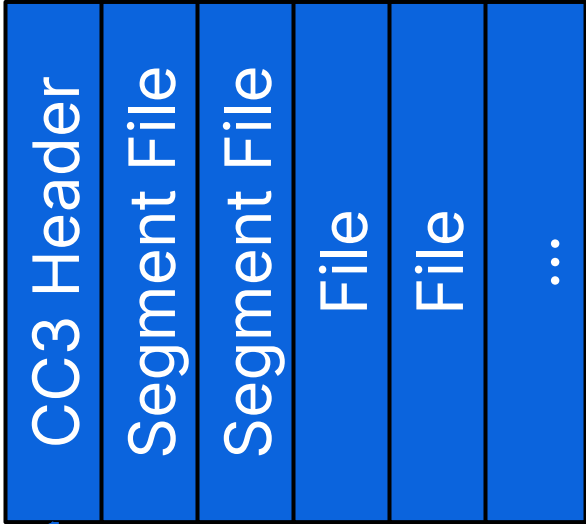
New: Batched Compound Commit

Minimizes PUT/GETs to the CSP to only BCC-level blob

BCC gen=1 blob file:



BCC gen=3 blob file:



References



New: Compound translog

- An indexing node combines operations of all shards into a single node-wide compound translog file.
- A shard recovery goes through the compound file to replay its shard's ops.
- Operations are acknowledged to the client after the upload.
- Indexing latency can go up due to that.
 - But a node hosting many busy shards should have minimal penalty.
 - We bound translog delay (~200ms).

New: Data safety

- Master is recognized through a “lease” file composed of:
 - Coordination term (used for consensus elections)
 - Node ID
 - Node left generation (increased whenever a node leaves the cluster)
- Atomic CAS on the lease to avoid split-brain
- Master keeps updating the cluster state in the object store
- Before acknowledging writes, node checks the lease file to ensure it has the up-to-date cluster state to know it's still owner of the shard.
- Garbage collection: Reference counts on BCC/translog blobs; search reports open generations; deletes only along safe paths.
 - Same check is done when we try to delete